

CI/CD Expert Patterns - GitLab Implementation

CI/CD Expert Patterns Analysis

Source: Chris Hayuk's Vibe Coding Templates (Python/GitHub Actions) **Adapted for:** TypeScript/Vite Blog (GitLab CI) **Date:** 2025-10-09

Key Expert Patterns Identified

1. Modular Workflow Design

Pattern: Separate workflows for different concerns instead of one monolithic file

Python/GitHub:

```
# ci.yml - Main integration
# github-actions-lint.yml - Dedicated linting
# github-actions-test.yml - Testing matrix
# github-actions-coverage.yml - Coverage tracking
# github-actions-docs.yml - Documentation
# github-actions-benchmark.yml - Performance
# github-actions-release.yml - Release automation
```

GitLab Adaptation: - Use multiple stages in single .gitlab-ci.yml - Jobs grouped by stage (lint, test, security, build, deploy) - Jobs can run in parallel within same stage

2. Matrix Testing Strategy

Pattern: Test across multiple versions and platforms

Python/GitHub:

```
strategy:
  fail-fast: false
matrix:
  os: [ubuntu-latest, macos-latest, windows-latest]
  python-version: ["3.10", "3.11", "3.12"]
```

TypeScript/GitLab:

```
test:
  parallel:
    matrix:
      - NODE_VERSION: ["18", "20", "22"]
        OS: ["alpine", "bullseye"]
```

Why it matters: - Catches environment-specific bugs - Ensures compatibility across versions - fail-fast: false shows ALL failures, not just first

3. Intelligent Caching

Pattern: Use lockfiles as cache keys for invalidation

Python/GitHub:

```
- uses: actions/cache@v3
  with:
    path: ~/.cache/uv
    key: ${{ runner.os }}-uv-${{ hashFiles('**/uv.lock') }}
    restore-keys: |
      ${{ runner.os }}-uv-
```

TypeScript/GitLab:

```
cache:
  key:
    files:
      - pnpm-lock.yaml # Invalidates when dependencies change
  paths:
    - .pnpm-store/
    - node_modules/
```

Why it matters: - Speeds up CI by 2-5x - Auto-invalidates when dependencies change - Restore-keys provide fallback caching

4. Security-First Approach

Pattern: Dedicated security scanning jobs

Python/GitHub:

```
security:
  steps:
    - name: Run Bandit security scan
      run: uv run bandit -r src/
      continue-on-error: true

    - name: Run Safety check
      run: uv run safety check
      continue-on-error: true
```

TypeScript/GitLab:

```
security-audit:
  stage: test
  script:
    - pnpm audit --prod
    - pnpm audit --audit-level=high
  allow_failure: true
```

Why it matters: - Finds known vulnerabilities early - continue-on-error/allow_failure = informational, doesn't block - Separate job = can be run on schedule

5. Multi-Layer Quality Gates

Pattern: Multiple independent quality checks

Python/GitHub:

```
lint:
  steps:
    - name: Run Ruff check (linting)
    - name: Run Ruff format check (formatting)
    - name: Run Black check (formatting)
    - name: Run MyPy (type checking)
    - name: Check import sorting
```

TypeScript/GitLab:

```
lint:
  script:
    - pnpm run lint          # ESLint
    - pnpm run format:check  # Prettier

typecheck:
  script:
    - pnpm exec tsc --noEmit # TypeScript strict mode
```

Why it matters: - Catches different classes of issues - Enforces consistent code style - Type safety prevents runtime errors

6. Coverage Tracking with Thresholds

Pattern: Track coverage and enforce minimum thresholds

Python/GitHub:

```
- name: Run tests with coverage
run: |
  uv run pytest --cov=src \
    --cov-report=xml \
    --cov-fail-under=80
```

TypeScript/GitLab:

```
test-coverage:
  script:
    - pnpm test:coverage --coverage.threshold.lines=80
  coverage: '/Lines\s*:\s*(\d+\.\d+)%/'
  artifacts:
    reports:
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
```

Why it matters: - Prevents coverage regression - GitLab shows coverage badge automatically - Forces writing tests for new code

7. Artifact Management

Pattern: Save build outputs and test results

Python/GitHub:

```
- name: Upload test results
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-results-${{ matrix.os }}-py${{ matrix.python-version }}
    path: test-results.xml
```

TypeScript/GitLab:

```
build:
  artifacts:
    name: "build-${CI_COMMIT_SHORT_SHA}"
    paths:
      - dist/
      - coverage/
    expire_in: 1 day
  reports:
    junit: test-results.xml
```

Why it matters: - Debugging failures in CI - Sharing builds between stages - Historical tracking

8. Smart Conditionals

Pattern: Run expensive steps only when needed

Python/GitHub:

```
- name: Upload coverage reports
  if: matrix.os == 'ubuntu-latest' && matrix.python-version == '3.11'
  uses: codecov/codecov-action@v4
```

TypeScript/GitLab:

```
upload-coverage:
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  only:
    - main
```

Why it matters: - Saves CI minutes - Prevents duplicate uploads - Runs heavy jobs only where needed

9. Environment Management

Pattern: Use environment variables for configuration

Python/GitHub:

```
env:
  PYTHON_VERSION: "3.11"
  UV_VERSION: "0.4.0"
```

TypeScript/GitLab:

```
variables:
  NODE_VERSION: "20"
  PNPM_VERSION: "latest"
```

10. Modern Tooling

Pattern: Use latest versions of actions/tools

Python/GitHub: - actions/checkout@v4 (not v3) - actions/setup-python@v5 - uv (modern fast package manager, not pip)

TypeScript/GitLab: - node:20-alpine (latest LTS, lightweight) - pnpm (faster than npm) - Latest Vercel CLI

Pattern Comparison: Python → TypeScript

Python Tool	TypeScript Equivalent	Purpose
uv	pnpm	Fast package manager
ruff	eslint	Linting
black	prettier	Formatting
mypy	tsc --noEmit	Type checking

pytest	vitest/jest	Testing
bandit	npm audit	Security scanning
safety	pnpm audit	Dependency vulnerabilities
interrogate	typedoc coverage	Documentation coverage
codecov	GitLab coverage	Coverage tracking

Key Takeaways for Commercial Projects

1. Fail Fast is Bad for CI

```
# BAD: Stops at first failure
fail-fast: true

# GOOD: Shows all failures
fail-fast: false
```

2. Cache Everything

```
# Use lockfile as cache key
key:
  files:
    - pnpm-lock.yaml
```

3. Matrix Testing is Essential

```
# Test multiple Node versions
parallel:
  matrix:
    - NODE_VERSION: ["18", "20", "22"]
```

4. Security Scans Should Be Informational

```
# Don't block on security warnings initially
allow_failure: true
```

5. Save Artifacts Always

```
# Even on failure
if: always()
```

Tools Recommendations

Python Projects: - Package manager: uv (10-100x faster than pip) - Linter: ruff (100x faster than pylint) - Formatter: black or ruff format - Type checker: mypy --strict

TypeScript Projects: - Package manager: pnpm (faster than npm/yarn) - Linter: eslint with TypeScript plugin - Formatter: prettier - Type checker: tsc --noEmit --strict

Expert Pattern Checklist

- ☐ Separate concerns into stages/jobs
- ☐ Matrix testing across versions
- ☐ Cache dependencies using lockfile
- ☐ Security scanning (allow_failure: true)
- ☐ Multi-layer quality gates (lint + format + typecheck)
- ☐ Coverage tracking with thresholds
- ☐ Artifact management for debugging
- ☐ Smart conditionals to save CI time
- ☐ Environment variables for configuration
- ☐ Modern tooling (latest versions)
- ☐ fail-fast: false for matrix jobs
- ☐ Meaningful job names
- ☐ Upload artifacts even on failure (if: always())
- ☐ Use official actions/images when possible